

函数代数与数学表达练习详解

一 同一规则的四种表示

文字可以写为“在合法工作量 $3/2$ 上先乘 2 再加 1 ”。公式计算为 $f(3/2) = 2 \times (3/2) + 1 = 3 + 1 = 4$ 。表格新增一行“输入 $3/2$, 输出 4 ”。程序表达式是 $y_hat = 2 * x + 1$, 令 x 为 `Fraction(3, 2)` 就能精确得到 4 。

若写成 $2 * (x + 1)$, 先做 $3/2 + 1 = 5/2$, 再乘 2 得 5 。差别来自运算顺序, 而不是浮点精度。即使两个程序都没有报错, 也不是同一规则。括号外的系数会乘给里面所有项。

二 定义域 陪域 与值域

定义域是 $[0, 4]$, 陪域是 $\mathbb{R}$, 值域是 $[1, 9]$ 。定义域说明允许输入谁, 陪域说明声明的输出空间, 值域说明实际可取得哪些输出。不能把只列了五个输入的采样表误当成函数只接收五个整数。

从 $0 \leq x \leq 4$ 推出 $1 \leq 2x + 1 \leq 9$, 只说明实际输出没有超出 $[1, 9]$ 。要证明该区间没有未达到的空缺, 还需对任意 $t \in [1, 9]$ 找到输入。令 $x = (t - 1)/2$, 它属于 $[0, 4]$, 并且 $f(x) = t$, 所以整个区间都在值域里。两部分合起来才得到相等。

三 域外候选解

$2x + 1 = 11$ 两边减 1 得 $2x = 10$, 再除以 2 得 $x = 5$ 。但 5 不在原定义域 $[0, 4]$ 内, 所以原受限函数没有满足条件的输入。代数表达式在 5 可以计算, 不表示受限函数允许接收它。

若另行定义 $F: \mathbb{R} \rightarrow \mathbb{R}$ 且 $F(x) = 2x + 1$, 则 $F(5) = 11$ 。这是新函数的解, 不能不加区分地写成原 f 的解。

四 含参数方程的分支

当 $a \neq 0$ 时, $ax + 1 = 5$ 化成 $ax = 4$, 得到 $x = 4/a$ 。最后要检查 $4/a \in [0, 4]$ 。由于分子 4 为正, a 为负会得到负输入而不合法; a 为正时 $4/a \leq 4$ 等价于 $a \geq 1$ 。所以若要求原输入范围, 恰在 $a \geq 1$ 时有合法解。

当 $a = 0$ 时, 原方程成为 $1 = 5$, 不成立, 因此无解。不能写 $x = 4/0$, 也不能把除以零称为无穷大解。若右边换成 1 , a 为 0 时才会出现所有合法 x 都满足的情况。

五 约分与被删掉的点

$x^2 - 4 = (x - 2)(x + 2)$ 。原式分母要求 $x \neq 2$, 所以

$$\frac{x^2 - 4}{x - 2} = x + 2, \quad x \neq 2.$$

x 为 2 时原式是零除以零，没有定义。化简式 $x + 2$ 在 2 等于 4，只表示它可以定义一个扩展函数；不能给原式合法赋值。图上可以把原式画成直线但在对应点留空洞。

六 复合顺序与定义域

输入 2 时, $f(2) = 5$, 所以 $g(f(2)) = 25$ 。反过来 $g(2) = 4$, 所以 $f(g(2)) = 9$ 。输入 3 时, 先 f 后 g 得 $7^2 = 49$; 先 g 后 f 得到中间数 9, 超出 f 定义域, 后者不合法。

$g \circ f$ 的定义域是 $[0, 4]$, 因为先通过 f , 而 g 接收所有实数。 $f \circ g$ 要求 $0 \leq x^2 \leq 4$, 得到 $[-2, 2]$ 。这两个复合是新的数值规则; 后一规则允许某些负输入, 不意味着虚构设备任务被批准接收负工作量。将函数当数学对象与将其用于现实任务需要分别说明。

七 幂与对数的定义

$2^{-3} = 1/2^3 = 1/8$ 。负指数取倒数, 不把结果变负。

$16^{1/2} = 4$, 因为取正平方根, 而 $4^2 = 16$ 。方程 $z^2 = 16$ 有正负两个解, 但主平方根本身选非负值。

$\log_2(1/8) = -3$, 因为 $2^{-3} = 1/8$ 。

$\log_{1/2} 4 = -2$, 因为 $(1/2)^{-2} = 4$ 。底数在 0 与 1 之间时, 真数大于 1 可以对应负对数值。

八 实数对数的资格

$\log_1 7$ 不合法: 底数为 1, 指数变化无法产生 7, 也无法形成通常的反向函数。 $\log_2 0$ 不合法: 2 的任意实数次幂都为正, 不会等于 0。 $\log_2(-4)$ 不合法: 负真数不属于本讲实数对数的定义域。

$\log_{1/2} 8$ 合法, 因为底数正且不为 1, 真数正。它等于 -3, 因为 $(1/2)^{-3} = 8$ 。合法底数不限于大于 1, 不应误删 0 到 1 之间的情况。

九 商规则与条件

条件为 $b > 0, b \neq 1, u > 0, v > 0$ 。令 $p = \log_b u, q = \log_b v$, 按定义有 $u = b^p, v = b^q$ 。第一步 $u/v = b^p/b^q$; 第二步用非零正底数指数商法则得到 b^{p-q} ; 第三步用对数定义反推 $\log_b(u/v) = p - q = \log_b u - \log_b v$ 。

若取 u 为 -2、 v 为 -8, 则 $u/v = 1/4$, 以 2 为底的对数左侧等于 -2, 合法。但右侧分别要求对 -2 与 -8 取实数对数, 均不合法。这不否定带正数条件的商规则, 而是说明不能删除条件。

十 求和与索引

r 的和为 $0 + 1 - 1 + 1 + 0 = 1$ 。各项平方之和为 $0 + 1 + 1 + 1 + 0 = 3$ 。和的平方为 $1^2 = 1$ 。平均平方差为 $3/5$ 。平方再求和与求和再平方是两种不同复合顺序。

数学第 4 条对应 Python 位置 3。这里 $xs[3]$ 为 3, $ys[3]$ 为 6, 预测为 7, 差为 1。输入数值恰好也为 3 是这组数据的巧合, 不是“Python 位置必须等于输入”的一般规则。改变样本顺序或输入取值后, 这种巧合就会消失。

十一 否定量词与共享参数

“对所有样本, 误差都为零”的否定是“至少有一个样本, 误差不为零”。第 2 条差为 1, 是一个见证; 第 3 或第 4 条也可以。无需证明每一条都错, 反而本例第 1 和第 5 条确实没有误差。

在 X 为 0、1、2, B 为 0、-2、-4 的例子中, 每个 x 分别选 $b = -2x$, 就能让 $2x + b = 0$ 。但一个共同 b 若同时适合 x 为 0 和 1, 就必须既等于 0 又等于 -2, 矛盾。量词从“对每个 x 存在 b ”改为“存在 b 对每个 x ”, 改变了参数能否依赖输入。

十二 五点一致与一个合法反例

$p(x) - f(x) = x(x-1)(x-2)(x-3)(x-4)$ 。在 0、1、2、3、4 中任取一个输入, 总有一个因子为零, 乘积也为零。因此五个整数点完全一致。

当 $x = 1/2$ 时, 有一个正因子和四个负因子, 结果为正; 分子绝对值相乘为 105, 分母相乘为 32, 故差为 $105/32$ 。 f 的值是 2, p 的值是 $2 + 105/32 = 169/32$ 。这是合法区间内的反例, 足以否定整个 $[0, 4]$ 上相同的主张。

如果定义域只声明为五元素集合 $\{0, 1, 2, 3, 4\}$, 全部五项精确核对足以证明这两个有限函数相同。注意论证对象已经变了: 有限集合与连续区间不是同一个定义域。

十三 归纳证明的四部分

命题为: 对任意正整数 n , $\sum_{i=1}^n (2i-1) = n^2$ 。

起点: n 为 1 时左边是 1, 右边也是 1。

归纳假设: 对某个任意正整数 k , 假定 $\sum_{i=1}^k (2i-1) = k^2$ 。这是用来证明条件推进的暂时前提, 不是提前宣布所有命题成立。

待证目标: $\sum_{i=1}^{k+1} (2i-1) = (k+1)^2$ 。

推进: 把左边最后一项分离, 得到 $\sum_{i=1}^k (2i-1) + 2(k+1) - 1$ 。在这里使用归纳假设, 将前 k 项的和换成 k^2 。继续展开成 $k^2 + 2k + 1$, 再利用平方展开式识别为 $(k+1)^2$ 。任意 k 都能这样推进, 结合起点得到所有正整数结论。

如果缺少起点，推进链没有入口；如果只检验 k 为 4 的步骤，就没有覆盖任意 k ；如果把待证的第 $k+1$ 条当假设代入，就形成循环论证。详细写出每行，正是为了检查这三种缺口。

十四 一百万次检查的范围

即使前一百万项全部精确通过，它直接证明的也只是“每个 n 属于 1 至 1000000 这个有限集合时，公式成立”。它没有穷尽所有正整数，因为还存在 1000001 以及无限多个更大整数。一般结论来自归纳证明中的任意推进，而非更多但仍有限的枚举。

如果程序使用浮点而不是精确整数，还要额外考虑计算误差与实现错误；即使使用精确整数，也必须保证程序实际检查的条件与目标命题一致。这里不是贬低测试，而是在给测试结论一个准确边界。

十五 精确输入与容差

`Fraction("0.1")` 读取十进制字符串，得到 $1/10$ 。`Fraction(0.1)` 读取 Python 已生成的二进制浮点数，其精确值通常是 $3602879701896397/36028797018963968$ ，与 $1/10$ 不同。`Fraction` 能精确保存所接收的数，却不能猜出这个数之前是否由近似生成。

对数的多数值不能用有限二进制小数精确表示，不同运算顺序还可能不同舍入。使用 `==` 可能把数值上已经足够接近的正确实现当成不一致。明确容差使本次核验有可解释的标准，但“落在容差内”只描述有限数值检查，不证明任意合法实数恒等式。

也不能随便把容差调大到任何错误都通过。应结合测试数值范围、浮点精度与所需误差尺度设定，并保留数学推导和极端情况限制。本实验采用相对与绝对容差各 10^{-12} ，不承诺它对所有将来问题都合适。

实验报告示范

我核对了受限函数 $f(x) = 2x + 1$ 的四种表达，并在输入 $3/2$ 得到精确预测 4。五条合成记录的平方差之和是 3，平均为 $3/5$ ；索引对应保持一致。程序按定义域拒绝非法输入，360 项有限对数核验符合明确容差，22 项边界检查通过。两个量词顺序给出不同结论，五点一致的函数在 $1/2$ 有精确差 $105/32$ 。奇数和公式的普遍结论由归纳证明支持， n 从 1 到 100 的程序检查只核验实现。此实验不证明真实设备预测能力，也未覆盖所有极端浮点输入。